24.1 浏览器架构概述

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160k tokens，输出 ~5k tokens（本节）

前几章我们看了 OpenClaw 怎么通过 Bash 工具让 AI Agent 在操作系统层面执行命令。但在现代 Web 时代，浏览器才是信息获取和交互的核心。OpenClaw 的浏览器控制子系统让 Agent 能像人一样浏览网页——导航、点击、填表、截图，甚至执行 JavaScript。这一节从全局视角看这个子系统的架构设计。

24.1.1 Playwright + CDP 混合方案

为什么需要两层?

OpenClaw 的浏览器控制用了**双层架构**：底层的 CDP（Chrome DevTools Protocol）和上层的 Playwright。

```
┌──────────────────────────────────────┐
│          OpenClaw Agent              │
│         (工具调用层)                  │
├──────────────────────────────────────┤
│     Playwright 高级 API 层           │
│  click / fill / snapshot / navigate │
│  页面状态管理 / 角色快照              │
├──────────────────────────────────────┤
│        CDP 低级 API 层               │
│  WebSocket → Chrome DevTools         │
│  截图 / JS 执行 / 标签页创建         │
├──────────────────────────────────────┤
│    Chrome / Chromium 浏览器实例      │
│  (OpenClaw 管理 或 用户已有)         │
└──────────────────────────────────────┘
```

> **衍生解释**：**CDP（Chrome DevTools Protocol）**是 Chrome 浏览器内置的远程调试协议。当 Chrome 以 `--remote-debugging-port=9222` 参数启动时，会在该端口上暴露一个 HTTP +

> WebSocket 接口。外部程序可以通过这个接口控制浏览器的几乎所有行为——导航页面、执行 JavaScript、截取屏幕截图、修改 DOM 等。它是所有浏览器自动化框架（Puppeteer、Playwright、Selenium 4）的底层基础。

> **衍生解释：Playwright** 是微软开发的浏览器自动化库。与直接使用 CDP 不同，Playwright 提供了更高级的抽象——自动等待元素可交互、智能选择器（通过 ARIA 角色、文本内容等定位元素）、跨浏览器支持（Chrome、Firefox、Safari）。对 AI Agent 来说，Playwright 的"角色快照（Aria Snapshot）"功能尤其重要——它能将页面的可访问性树（Accessibility Tree）转换为 Agent 可读的文本描述。

混合方案的设计理由：

场景	使用层	原因
截图	CDP	直接调用 `Page.captureScreenshot`，零额外开销
执行 JS	CDP	直接调用 `Runtime.evaluate`，无需 Playwright 上下文
创建标签页	CDP	`Target.createTarget` 更底层可靠
点击/填表	Playwright	自动等待、智能定位、错误恢复
页面快照	Playwright	ARIA 快照是 Playwright 独有功能
网络监控	Playwright	Request/Response 事件更易使用

24.1.2 Chrome 实例管理

浏览器发现与启动

`chrome.ts` 负责发现系统中的 Chrome/Chromium 浏览器并管理其生命周期：

```typescript
// src/browser/chrome.ts — RunningChrome 类型
type RunningChrome = {
  pid: number;                        // 进程 ID
  exe: BrowserExecutable;             // 浏览器可执行文件信息
  userDataDir: string;                // 用户数据目录
  cdpPort: number;                    // CDP 调试端口
  startedAt: number;                  // 启动时间
  proc: ChildProcessWithoutNullStreams; // 子进程句柄
};
```

浏览器可执行文件发现

`chrome.executables.ts` 为三大平台（macOS、Linux、Windows）提供了浏览器发现逻辑：

```
macOS:
  1. /Applications/Google Chrome.app/Contents/MacOS/Google Chrome
  2. /Applications/Brave Browser.app/Contents/MacOS/Brave Browser
  3. /Applications/Microsoft Edge.app/Contents/MacOS/Microsoft Edge
  4. /Applications/Chromium.app/Contents/MacOS/Chromium

Linux:
  1. google-chrome / google-chrome-stable (PATH 搜索)
  2. chromium / chromium-browser
  3. brave-browser
  4. microsoft-edge

Windows:
  1. %PROGRAMFILES%\Google\Chrome\Application\chrome.exe
  2. %LOCALAPPDATA%\Google\Chrome\Application\chrome.exe
  3. Brave、Edge 类似路径
```

启动流程

`launchOpenClawChrome()` 的启动流程包含多个阶段：

```
1. 端口检查 → ensurePortAvailable(cdpPort)
2. 发现浏览器 → resolveBrowserExecutable()
3. 用户数据目录 → ~/.openclaw/browser/<profile>/user-data/
4. 首次引导（可选）:
   └─ spawn Chrome → 等待配置文件生成 → 关闭
5. 配置文件装饰:
   └─ 设置 OpenClaw 主题颜色、名称
6. 正式启动:
   └─ spawn Chrome with --remote-debugging-port=<cdpPort>
7. 等待 CDP 就绪:
   └─ 轮询 /json/version（最长 15 秒）
```

Chrome 启动参数经过精心选择：

```
const args = [
  `--remote-debugging-port=${profile.cdpPort}`,  // CDP 端口
  `--user-data-dir=${userDataDir}`,               // 隔离的数据目录
  "--no-first-run",                  // 跳过首次运行向导
  "--no-default-browser-check",      // 不检查默认浏览器
  "--disable-sync",                  // 禁用 Google 同步
  "--disable-background-networking", // 减少网络噪声
  "--disable-component-update",      // 禁用组件更新
  "--disable-features=Translate,MediaRouter",
  "--disable-session-crashed-bubble",
  "--hide-crash-restore-bubble",
  "--password-store=basic",          // 简化密码存储
  "about:blank",                     // 打开空白页（确保至少有一个 target）
];
```

CDP 就绪检测

```
// src/browser/chrome.ts — 健康检查
export async function isChromeReachable(cdpUrl: string, timeoutMs = 500) {
  const version = await fetchChromeVersion(cdpUrl, timeoutMs);
  return Boolean(version);
}

// 更严格的检查：HTTP + WebSocket 都通
export async function isChromeCdpReady(cdpUrl, timeoutMs, handshakeTimeoutMs) {
  const wsUrl = await getChromeWebSocketUrl(cdpUrl, timeoutMs);
  if (!wsUrl) return false;
  return await canOpenWebSocket(wsUrl, handshakeTimeoutMs);
}
```

两级检查确保 Chrome 不仅启动了 HTTP 服务，WebSocket 连接也能正常建立。

24.1.3 配置文件（Profiles）管理

多配置文件设计

OpenClaw 支持同时管理多个浏览器配置文件，每个配置文件有独立的 CDP 端口和用户数据目录：

```typescript
// src/browser/config.ts — 解析后的配置
type ResolvedBrowserConfig = {
  enabled: boolean;
  controlPort: number;          // 浏览器控制服务端口
  cdpProtocol: "http" | "https";
  cdpHost: string;
  headless: boolean;            // 无头模式
  noSandbox: boolean;           // 是否禁用 Chrome 沙箱
  attachOnly: boolean;          // 仅连接已有浏览器
  defaultProfile: string;       // 默认配置文件名
  profiles: Record<string, BrowserProfileConfig>;
};

type ResolvedBrowserProfile = {
  name: string;                 // 配置文件名（如 "openclaw"、"chrome"）
  cdpPort: number;              // CDP 端口（18800–18899）
  cdpUrl: string;               // 如 "http://127.0.0.1:18800"
  cdpIsLoopback: boolean;       // 是否本地（非远程）
  color: string;                // 配置文件主题色
  driver: "openclaw" | "extension"; // 控制方式
};
```

CDP 端口分配

```typescript
// src/browser/profiles.ts — 端口范围
export const CDP_PORT_RANGE_START = 18800;
export const CDP_PORT_RANGE_END   = 18899;  // 最多 100 个配置文件

// 保留端口（不用于 CDP）：
//   18789 – Gateway WebSocket
//   18790 – Bridge
//   18791 – 浏览器控制服务
//   18792–18799 – 保留（Canvas 在 18793）
```

`allocateCdpPort()` 从范围中找到第一个未被使用的端口。已使用的端口从所有配置文件中收集：

```
export function allocateCdpPort(usedPorts: Set<number>): number | null {
  for (let port = CDP_PORT_RANGE_START; port <= CDP_PORT_RANGE_END; port++) {
    if (!usedPorts.has(port)) return port;
  }
  return null;
}
```

配置文件主题色

每个配置文件有一个可视化标识色，用于在 Chrome 的标题栏上区分不同实例：

```
// src/browser/profiles.ts — 预定义色板
export const PROFILE_COLORS = [
  "#FF4500",  // 橘红（openclaw 默认）
  "#0066CC",  // 蓝
  "#00AA00",  // 绿
  "#9933FF",  // 紫
  "#FF6699",  // 粉
  "#00CCCC",  // 青
  "#FF9900",  // 橙
  "#6666FF",  // 靛
  "#CC3366",  // 品红
  "#339966",  // 碧绿
];
```

`decorateOpenClawProfile()` 修改 Chrome 的 `Preferences` 和 `Local State` 文件，注入配置文件名和主题色。这样用户一眼就能辨认出哪个 Chrome 窗口是 OpenClaw 控制的。

两种驱动模式

驱动	场景	原理
`openclaw`	OpenClaw 自己启动和管理 Chrome	完全控制生命周期
`extension`	用户已有 Chrome，安装了 OpenClaw 扩展	通过 Chrome Extension Relay 中继 CDP 命令

`extension` 模式解决了一个实际问题：用户可能已经在用 Chrome 并登录了各种网站。与其启动一个全新的 Chrome 实例（没有 cookie、没有登录状态），不如复用用户现有的浏览器——只需安装一个 OpenClaw 扩展来建立控制通道。

24.1.4 配置常量

```
// src/browser/constants.ts
export const DEFAULT_OPENCLAW_BROWSER_ENABLED = true;
export const DEFAULT_BROWSER_EVALUATE_ENABLED = true;
export const DEFAULT_OPENCLAW_BROWSER_COLOR = "#FF4500";
export const DEFAULT_OPENCLAW_BROWSER_PROFILE_NAME = "openclaw";
export const DEFAULT_BROWSER_DEFAULT_PROFILE_NAME = "chrome";
export const DEFAULT_AI_SNAPSHOT_MAX_CHARS = 80_000;
export const DEFAULT_AI_SNAPSHOT_EFFICIENT_MAX_CHARS = 10_000;
export const DEFAULT_AI_SNAPSHOT_EFFICIENT_DEPTH = 6;
```

`DEFAULT_AI_SNAPSHOT_MAX_CHARS = 80,000` 和 `DEFAULT_AI_SNAPSHOT_EFFICIENT_MAX_CHARS = 10,000` 控制页面快照发送给 LLM 的文本长度上限——页面越复杂，快照越大，但过大的快照会消耗过多的 context window。

本节小结

1. **双层架构**：CDP 用于低级操作（截图、JS 执行、标签页管理），Playwright 用于高级操作（点击、填表、页面快照）
2. **Chrome 生命周期管理**：自动发现浏览器可执行文件（支持 Chrome/Brave/Edge/Chromium）、创建隔离的用户数据目录、首次引导配置文件、等待 CDP 就绪
3. **多配置文件**：每个配置文件分配独立的 CDP 端口（18800-18899 范围）、主题色、用户数据目录
4. **两种驱动模式**：`openclaw` 模式自行启动 Chrome，`extension` 模式通过扩展中继复用用户已有的浏览器
5. **端口规划**清晰——Gateway、Bridge、浏览器控制、Canvas 各有保留端口，CDP 端口池最多支持 100 个配置文件